# DeePC (Data-Enabled Predictive Control) for Reinforcement Learning Transferability on a 7 DoF Robot

Leo Noth, Lucas Gimeno, Mirko Meboldt

***Abstract*—Reinforcement learning (RL) policies trained in simulation often suffer performance degradation when deployed on real robotic systems due to discrepancies between simulated and physical dynamics. Existing approaches typically address this sim2real gap through domain randomization or policy adaptation, which increase training complexity and require additional learning stages. This work proposes a control-theoretic alternative that regulates the sim2real discrepancy during deployment. The gap between simulated and real robot behavior is modeled as an approximately linear time-invariant system and compensated online using Data-Enabled Predictive Control (DeePC). With this active regulation, the real system is steered toward simulator-consistent behavior without modifying the trained policy. Experimental validation on a 7-DoF Franka Emika Panda robotic arm demonstrates improved policy transferability in a repeated reach task with randomized targets. The proposed method reduces terminal pose errors and significantly improves execution consistency compared to zero-shot policy deployment.The results suggest that regulating the sim2real gap during execution provides a practical alternative to learning-based adaptation for improving real-world RL deployment.**

***Index Terms*—Data enabled predictive control, sim2real gap, reinforcement learning, robotics, online friction compensation, friction mismatch modeling, blended control**

## I. Introduction

Deploying reinforcement learning policies (RL) trained in simulation on real systems remains a challenging task [1] [2] [3]. Favorable success rates and expected behavior are often only observed in simulation [4]. This sim2real gap problem is subject to active research with popular methods including domain randomization [5] and adaptation [6], system identification [7] and residual learning [8] [9] or a combination thereof.

In the case of domain randomization, the training environment is largely extended by randomly altering physical parameters to increase policy robustness against artificially induced variability. This comes with the tradeoff of higher training efforts, overly conservative behavior, and reduced sample efficiency [10] [11].

Domain adaptation and residual learning address different aspects of the sim2real gap problem, namely dynamics mismatch and observation distribution mismatch, but share similar practical limitations. Both rely on additional machine learning steps ahead of deployment and have limited scalability [1] [12]. Introducing auxiliary learning components increases system complexity and depends on careful high dimensional tuning [13].

Treating the sim2real gap problem as a secondary learning problem might not be a sustainable technique. Avoiding the need for environment randomization or subsequent discrepancy learning could enable a more streamlined RL framework. Similar in spirit to system identification, the idea introduced in this paper approaches the sim2real gap from the perspective of the physical system rather than modifying the policy itself.

A mechanism that actively forces the real world experience to be closer to the familiar simulator representation would reduce the dependency on the policy's robustness. Trained policies are optimized for the specific underlying Markov Decision Process (MDP), after which a task has been modeled [14]. A shift in transition dynamics between simulation and reality represents a distributional shift degrading performance upon transfer [15]. Actively accounting for this shift online preserves the expected MDP correspondence for which the policy was optimized [16]. Addressing the sim2real gap from the side of the real system, not from the training domain, with a control-theoretic approach is at the core of this work.

Related works treat the sim2real gap as a control problem and try to make use of familiar mechanisms such as error integration and feedback regulation to bring the real and simulated dynamics closer together . Promising results were found by adopting PLAI (Policy Level Action Integration) [17] [18] [19]. This algorithm accumulates residual error between commanded and achieved states, compensating for unmodeled dynamics, thus enabling zero shot policy transfer [20] [21]. PLAI essentially sees the frozen policy as a feedforward plant model and closes the loop with integral feedback at deployment.

Other promising findings utilize the simulator itself online [22]. A recent publication argues that if the real system simply follows the simulated system, the gap could be completely eliminated [23]. The policy lives completely in the simulation, so its governing MDP does not experience any shift in distribution. The real system is then tied to the simulated version by a high stiffness PD controller. Subsequently, performance is increased in a wide range of tasks. In particular, the quick reinforcement learning transferability was mentioned as a key benefit, allowing rapid policy prototyping. As adaptability from the policy's side

This paper was produced by the IEEE Publication Technology Group. They are in Piscataway, NJ.

Manuscript received April 19, 2021; revised August 16, 2021.

0000–0000/00$00.00 © 2021 IEEE

is no longer necessary, task specific tuning efforts can be skipped. Running the simulator online during real world deployment provides access to simulator-consistent state information, which can be exploited to reduce discrepancies between simulated and physical system behavior. [24] [25].

In this work, the sim2real gap is modeled as an LTI approximation based on physical joint differences between simulator and real system. Running the simulator online gives access to the online gap signal.

DeePC (Data-Enabled Predictive Control) [26] is employed as a promising control algorithm to regulate the gap signal as it enables simultaneous system identification and optimal control. This approach is particularly well suited to tasks, where rich data is available, an explicit model is unavailable and system dynamics are sufficiently slow to permit online optimization. Additionally, satisfying predefined torque, velocity, or energy constraints allows for safer operation, such as in human robot collaboration (HRC), contact-rich or otherwise safety-critical tasks [27].

Especially in the context of HRC, compliant actuation is essential to ensure safety [28]. However, such compliance increases sensitivity to unmodeled dynamics, load variations, and contact disturbances, further amplifying the sim2real gap. Compared to stiff industrial manipulators, compliant robotic arms exhibit more pronounced mismatch and tracking errors [29], which degrades the transferability of RL policies. Addressing this increased dynamics mismatch in a principled manner is therefore particularly relevant for HRC scenarios.

## II. Setup and Methodology

### A. Sim2Real Gap as an LTI System

The basis of this work is a reformulation of the sim2real gap as a control problem. The underlying system of which is taken to be the online physical joint differences between the simulator and the real system experienced during deployment. The joint positions, $q_k^r \in \mathbb{R}^n$ for the real system and $q_k^s \in \mathbb{R}^n$ for the simulated system, respectively, result from running a RL policy $\pi$ on a $n$ joint system and evolve accordingly.

$$u_k = \pi(o_k), \qquad q_{k+1}^{(\cdot)} = f^{(\cdot)}\big(q_k^{(\cdot)},\, u_k^{(\cdot)}\big), \quad (\cdot) \in \{r, s\} \tag{1}$$

Here, $o_k$ represents the input to the policy.

The joint differences $e_k := q_k^r - q_k^s$ are in large part the remaining dynamics mismatch between the simulated and the real robotic arm subject to the same RL policy. In addition, it is assumed that these residual dynamics can be approximated as a linear time invariant (LTI) system [30], bringing the sim2real gap system to the following form.

$$e_{k+1} = Ae_k + B\,\delta u_k + w_k, \qquad y_k = e_k \tag{2}$$

where $\delta u_k$ is an additive corrective input that actively regulates the system, $A$ and $B$ are constant unknown model parameters and $w_k$ captures any unmodeled disturbances and nonlinearities. The LTI approximation aligns with the theoretical basis of DeePC and allows it to be applied effectively based on input-output behavior. Controlling this system via $\delta u_k$ as the optimal input derived using DeePC and driving it to zero while operating actively closes the gap.

$$\lim_{k\to\infty} y_k = 0 \iff \text{simulated and real trajectories match} \tag{3}$$

### B. Data-Enabled Predictive Control

The proposed use of DeePC [26] requires a control framework capable of regulating an unknown system using measured input-output data, without relying on an explicit parametric model. DeePC is built on behavioral system theory, wherein future trajectories of LTI systems can be represented using previously collected input-output data arranged in a Hankel matrix. Under the assumption that the system is persistently excited and well approximated by LTI dynamics, DeePC enables model predictive control using data alone. In the following the DeePC formulation relevant to this work is summarized.

Given a series of (potentially multi-dimensional) input-output data $\{u_k, y_k\}_{k=0}^{T-1}$ collected from the system, where $u_k \in \mathbb{R}^m$ and $y_k \in \mathbb{R}^p$ denote the input and output at time step $k$, respectively, a block Hankel matrix of depth $L$ is constructed as

$$\mathcal{H}_L(u) = \begin{bmatrix} u_0 & u_1 & \cdots & u_{T-L} \\ u_1 & u_2 & \cdots & u_{T-L+1} \\ \vdots & \vdots & \ddots & \vdots \\ u_{L-1} & u_L & \cdots & u_{T-1} \end{bmatrix}, \tag{4}$$

with an analogous construction for the output data $\mathcal{H}_L(y)$. The combined input-output Hankel matrix is given by

$$\begin{bmatrix} \mathcal{H}_L(u) \\ \mathcal{H}_L(y) \end{bmatrix}. \tag{5}$$

Under the stated assumptions, for any trajectory $(u, y)$ of length $L$, there exists a coefficient vector $g \in \mathbb{R}^{T-L+1}$ such that

$$\begin{bmatrix} \mathcal{H}_L(u) \\ \mathcal{H}_L(y) \end{bmatrix} g = \begin{bmatrix} u \\ y \end{bmatrix}. \tag{6}$$

For predictive control, the Hankel matrix is partitioned into past and future components with respect to a prediction horizon $N$, where $L = T_{\text{ini}} + N$, giving

$$\mathcal{H}_L(u) = \begin{bmatrix} U_p \\ U_f \end{bmatrix}, \quad \mathcal{H}_L(y) = \begin{bmatrix} Y_p \\ Y_f \end{bmatrix}. \tag{7}$$

where the subscripts $p$ and $f$ denote the past and future components. Given a measured past trajectory $(u_{\text{ini}}, y_{\text{ini}})$ of length $T_{\text{ini}}$, the DeePC optimization problem is formulated as

$$\begin{aligned} \min_{g,\,u,\,y,\,\sigma_y} \quad & \sum_{k=0}^{N-1} \big(\|y_k - r_{t+k}\|_Q^2 + \|u_k\|_R^2\big) + \lambda_g \|g\|_2^2 + \lambda_y \|\sigma_y\|_2^2 \\ \text{s.t.} \quad & \begin{pmatrix} U_p \\ Y_p \\ U_f \\ Y_f \end{pmatrix} g = \begin{pmatrix} u_{\text{ini}} \\ y_{\text{ini}} \\ u \\ y \end{pmatrix} + \begin{pmatrix} 0 \\ \sigma_y \\ 0 \\ 0 \end{pmatrix} \\ & u_k \in \mathcal{U}, \quad \forall k \in \{0, \ldots, N-1\}, \\ & y_k \in \mathcal{Y}, \quad \forall k \in \{0, \ldots, N-1\} \end{aligned} \tag{8}$$

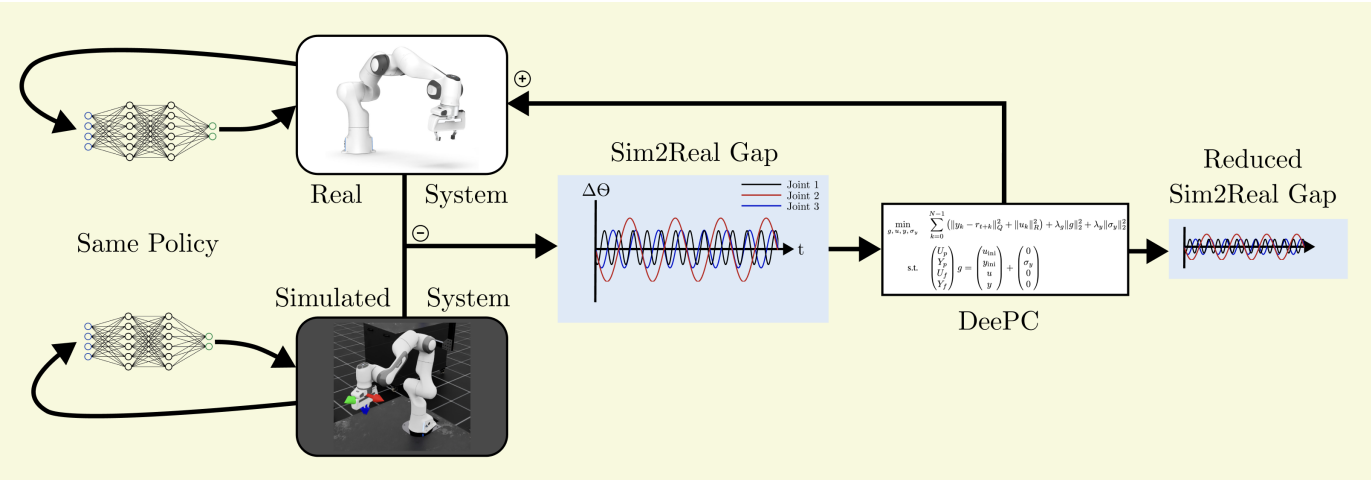


Fig. 1. The setup used to improve RL policy transferability with DeePC. The real and simulated system run on their own in parallel, while the resulting gap is actively accounted for in a feedback control loop.

Here, $r$ denotes the reference trajectory, $Q \succeq 0$ and $R \succ 0$ are weighting matrices, and $\lambda_g$ and $\lambda_y$ are regularization parameters used to improve robustness in the presence of noise and finite data. The sets $\mathcal{U}$ and $\mathcal{Y}$ encode input and output constraints, respectively. Adjacent constraints, such as torque or velocity limits, can also be part of this optimization problem allowing for straight forward HRC compatible dynamics.

### C. Online DeePC Formulation for Sim-to-Real Gap Closure

The implementation of DeePC for the transferability of RL policies consists of a real robotic system, its simulated version, a trained policy, and the derived gap signal (system equation 2), which is subsequently used to close the sim2real gap. The overview of the conceptual setup is depicted in Figure 1.

The configuration has two separate instances of the same previously simulation-trained policy running simultaneously. The sim2real gap is actively regulated by a feedback loop that commands torque to the real system based on the online optimization problem shown in equation 8. As a result, the sim2real gap is reduced.

In particular, the input of the DeePC algorithm is taken to be $e_k$ while the output $\tau_k^{DeePC} := \delta u_k \in \mathbb{R}^n$ corresponds to a corrective control signal. The total torque applied to the real robotic system then consists of two parts: $\tau_k^r = \tau_k^{policy} + \tau_k^{DeePC}$. Equivalent to a model predictive control (MPC) formulation, DeePC runs in a receding horizon fashion. Mention how the ability to set constraints to the DeePC gives HRC guarantees. (Torque, Velocity, Energy or Momentum limits.)

### D. Experimental Platform and Implementation

The experimental validation is conducted on a 7 degree of freedom (DoF) Franka Emika Panda robotic arm [31]. This robot can be operated using torque commands, which allows for the implementation of custom low level controllers. Operating with low stiffness contributes to compliant HRC behavior. Torques are applied via the Franka Control Interface (FCI), which provides real-time access at 1kHz.

The simulation environment is implemented in IsaacLab [32] using the official Franka Panda Emika URDF model. To achieve a real-time factor of $\geq 1$, the physics engine (PhysX) is running at 500 Hz on a Nvidia RTX 4090. To match this, FCI has been set to the same communication frequency. The IsaacLab robotics learning framework provides the ability to implement handcrafted low level controllers. Carefully matching the real and simulated system, both low level controllers were set to the same stiffness and update frequency to ensure there is no architectural mismatch and the observed sim2real gap originates solely from residual dynamics and unmodeled physical effects rather than differences in control implementation.

To effectively solve the online optimization problem that DeePC poses, cvxpy [33] is used with the MOSEK solver [34]. Running on an AMD Ryzen 9 7900 CPU, a solve time below 100 ms was targeted, letting the DeePC algorithm run at a frequency of 10 Hz. To collect sufficiently rich sim2real gap data for successful online system identification, the underlying Hankel matrix was constructed using $T = 300$ data points, where $T$ denotes the total length of the collected input output dataset. A simple P-controller was used to produce the corrective torque signal as input while the sim2real gap $e_k$ was recorded as output (2). The resulting dimension of the Hankel matrix is $280 \times 281$, corresponding to a past horizon $T_{ini}$ of 5 and a future prediction horizon $N$ of 15. This fully stacked Hankel matrix size (8) can be calculated accordingly:

$$L = T_{\text{ini}} + N$$

$$\begin{bmatrix} U_p \\ Y_p \\ U_f \\ Y_f \end{bmatrix} \in \mathbb{R}^{(m+p)L \times (T-L+1)}$$

An appropriate sizing is critical as it is inherently bound to the controller's performance in terms of prediction accuracy, response time, numerical conditioning, and closed-loop robustness. The DeePC tuning parameters used in the experiments

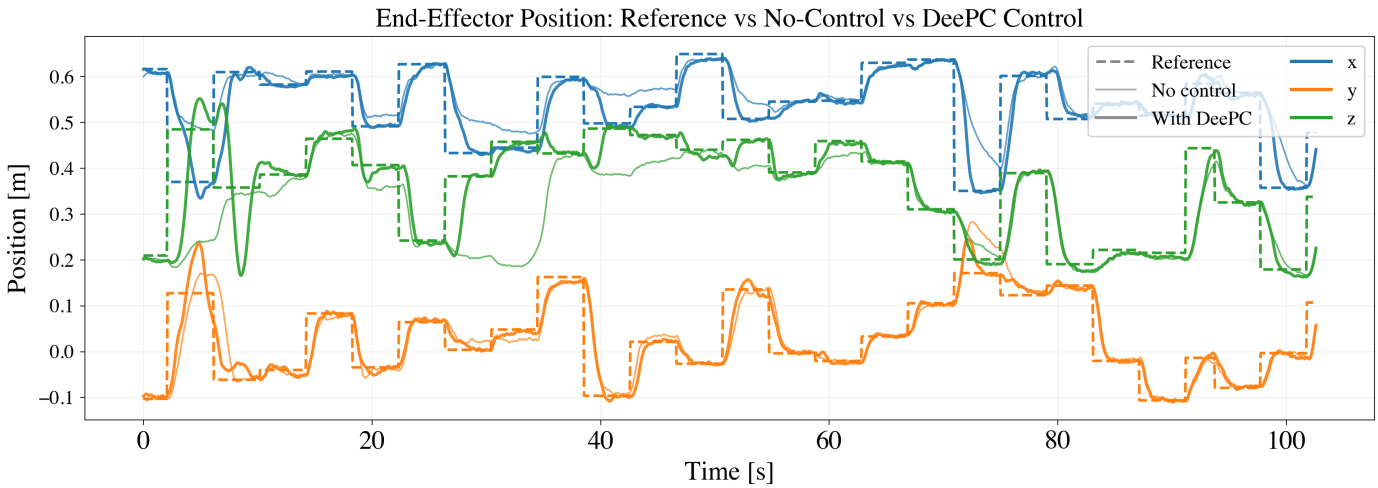


Fig. 2. The $(x, y, z)$ end effector behavior of the RL policy subject to 25 consecutive reference signals with and without DeePC sim2real gap control.

are summarized in Table IV (Appendix A).

The reinforcement learning task at hand is the classical reach task. The objective of the robot is to reach a predefined target pose $x_{ref} \in \mathbb{R}^7$ within a set time horizon of 4 seconds. The pose is parametrized by a Cartesian position $(x, y, z) \in \mathbb{R}^3$ and an orientation represented as a unit quaternion $q \in \mathbb{R}^4$. This reach task is continuously repeated during operation, forming a cyclic trajectory generated by the learned RL policy. Importantly, the objective of the policy depends fully on the reward formulation during training, which in this case penalizes excessive velocities and encourages compliant motion suitable for HRC scenarios. Rather than reaching the target pose as quickly as possible, smooth and stable motion is incentivized. The task is chosen to be structurally simple and could be solved using conventional inverse kinematics with joint-space trajectory tracking methods. Thus, the purpose of this experiment is not to showcase task specific complexity, but to provide a controlled setting in which the sim2real gap can be isolated and evaluated.

Accordingly, the RL policy receives joint positions, joint velocities, and the goal pose as observations and outputs desired joint positions for the seven actuated joints. These position commands are tracked by a low-level joint-space PD controller. No domain randomization is introduced during training, and the trained policy remains fixed throughout all experiments, being an unchanged part of the control loop.

## III. Results and Discussions

To assess the effectiveness of DeePC on the transferability of the RL policy and its ability to improve the experienced sim2real gap, the reach task is repeated continuously with randomized input poses. This results in a generated trajectory that tracks a given pose every 4 seconds using only the learned kinematics. RL policy inference is running at $20Hz$, while the low level PD controller tracks the produced desired joint positions at $500Hz$. This is compared to the robot's behavior while being assisted by the active online sim2real gap control. In this case, the robot receives the correctional torque inputs according to the DeePC Setup at a frequency of $10Hz$.

TABLE I
Normalized joint differences of zero-shot policy execution and DeePC gap control.

| | Mean | Std | Median | RMSE | Max | Min |
|---|---|---|---|---|---|---|
| Zero Shot | 0.5603 | 0.5629 | 0.3395 | 0.7943 | 2.1541 | 0.0303 |
| DeePC | 0.2620 | 0.2553 | 0.1581 | 0.3658 | 1.3059 | 0.0189 |
| Improvement | 53.2% | 54.6% | 53.4% | 53.9% | 39.4% | 37.6% |

This comparison of tracking performance can be seen in Figure 2. The plot shows only the end effector's $(x, y, z)$ Cartesian coordinates and leaves out the unit quaternion for clarity. The reference tracking improves visibly when the deepc controller is active. In particular, the uncorrected policy tends to miss certain poses entirely (see the $x$ and $y$ reference at $\sim$ 30 seconds) while also experiencing an overall poorer tracking. These pose misses are repeatably noticeable and occurred in the majority of experimental runs. Furthermore, once a pose is missed, it has a tendency to reach its consecutive pose to a lesser extent.

The observed divergence between the zero-shot and DeePC-assisted trajectories can be interpreted as a distributional shift in the underlying Markov Decision Process (MDP).

Once the execution leaves the nominal training distribution, subsequent action selections are conditioned on altered states, which may further amplify the deviation. This sequential propagation of error can explain the observed pose misses and the cumulative tracking degradation visible in Figure 2 especially on the $x$ and $y$ axes.

In contrast, DeePC-based gap regulation accounts for residual dynamics discrepancies in real time, thereby reducing the distributional shift between simulation and reality. By actively steering the physical system toward simulator-like behavior, the policy operates within a state–action operating domain closer to its training distribution, resulting in improved trajectory consistency and reduced pose deviations.

To further quantify the effect of the proposed gap regulation, the magnitude of the defined difference signal (2) is shown in Figure 3. The plotted results represent the Euclidean norm of the joint differences between the simulated and real robot

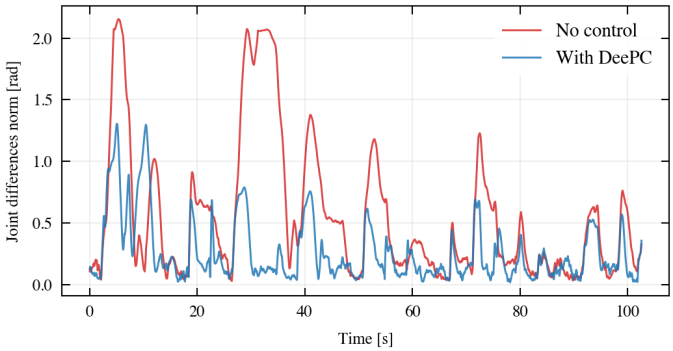


Fig. 3. Normalized joint differences $|e_k|$ during 25 reach task execution.

during the same set of reference signals as in Figure 2. When DeePC is active, the magnitude of these deviations is consistently reduced, and larger peaks are attenuated. This shows active sim2real gap compensation. The reduction of $|e_k|$ over these 25 consecutive runs can be seen in Table I.

Individual joints exhibit different magnitudes of mismatch. Generally, the friction of the real robot was found to be higher than in simulation, but not necessarily on all its joints. Higher total torque loads also scale up any difference in modeled and actual experienced friction. In practice, rather than normalizing the input data, the DeePC algorithm accounts for this with diagonal weights in the cost matrix $Q$.

Extending the experimental validation of this method to a larger set of 100 randomized target poses, the reach task was executed repeatably. While the statistics presented in Table I quantify the overall reduction of the sim2real gap, they do not directly reflect the success of the reach task itself. The RL policy used in this work was trained with a velocity penalty to encourage smooth and compliant motion suitable for HRC. As a result, conventional tracking metrics, such as full trajectory RMSE are not fully representative of task performance, since the objective of the policy is not to minimize instantaneous tracking error but rather to reach the desired pose within the given time horizon.

To better capture task level performance, a terminal error metric is introduced. For each reach task, the terminal RMSE is defined as the Euclidean root mean square error between the commanded and achieved end effector positions at the end of the motion. To reduce the influence of measurement noise, the final 25 samples of the reach phase are considered when computing this value. The Cartesian position components $x, y, z$ are used for the metric, as these provide the most interpretable measure of task completion for the reach scenario.

The terminal RMSE for a single reach task is computed over the final $K$ samples as:

$$\mathrm{RMSE}_{\mathrm{term}} = \sqrt{\frac{1}{K} \sum_{i=T-K+1}^{T} \left\| p_i - p_i^{\mathrm{ref}} \right\|_2^2}, \qquad p_i := \begin{bmatrix} x_i \\ y_i \\ z_i \end{bmatrix}. \tag{9}$$

Using this definition, the experiment yields 100 terminal RMSE values for both the zero-shot execution and the DeePC assisted scenario. The resulting distributions are shown in Figure 4. The DeePC assisted configuration consistently achieves lower terminal errors and exhibits reduced variance across trials, indicating more reliable task completion. This behavior is consistent with the earlier observations: while DeePC does not modify the underlying policy itself, it improves execution by regulating residual dynamics discrepancies, thereby maintaining system trajectories closer to the simulator consistent state distribution on which the policy was trained. Descriptive statistics of the terminal RMSE distributions are summarized in Table II. Across all metrics, the DeePC assisted configuration achieves lower terminal errors compared to zero-shot execution. In particular, the mean terminal RMSE is reduced from $0.044m$ to $0.0181m$, corresponding to an improvement of 58.7%. Increased accuracy and consistency can be deduced from the lower observed median and variance related measures.

TABLE II
DESCRIPTIVE STATISTICS OF TERMINAL RMSE PER CONDITION N=100.

| | Mean [m] | Std [m] | Median [m] | Min [m] | Max [m] |
|---|---|---|---|---|---|
| Zero-Shot | 0.0440 | 0.0507 | 0.0215 | 0.0033 | 0.2321 |
| DeePC | 0.0181 | 0.0267 | 0.0137 | 0.0022 | 0.2035 |
| Improvement | 58.7% | 47.3% | 36.3% | 33.3% | 12.3% |

TABLE III
STATISTICAL TEST RESULTS FOR TERMINAL RMSE COMPARISON.

| Welch $t$ | df | $p$ (Welch) | Cohen's $d$ | $\Delta\mu$ [m] |
|---|---|---|---|---|
| 4.502 | 150.0 | $1.34 \times 10^{-5}$ | 0.637 (medium) | 0.0258 |

Assessing the statistical significance of these improvements, a two sample Welch's t-test was conducted on the terminal RMSE samples obtained for both settings. Welch's t-test is chosen to account for unequal variances between the two distributions. The resulting statistics are reported in Table III. A p-value of $p = 1.34 \cdot 10^{-5}$ indicates the reduction in terminal error is statistically significant. The effect size, measured by Cohen's $d = 0.637$, indicates a medium effect according to the conventional interpretation.

The difference in mean terminal RMSE $\Delta\mu[m]$ indicates improved consistency rather than strictly higher accuracy. Although the results show an average improvement of 2.6 cm, this does not mean every individual estimate is 2.6 cm more accurate. In some cases the improvement may be smaller or absent. Instead, the results suggest that the method produces more stable and reliable outcomes across trials, as can be seen by the references that are missed by the uncompensated system, whereas the compensated system follows all of them.

## IV. CONCLUSION AND OUTLOOK

This work presented a control-theoretic approach to improving the transferability of RL-policies from simulation to real robotic systems. Instead of modifying the training environment or adapting the policy after deployment, the sim2real gap is treated as an LTI system. Regulating this gap online with DeePC actively drives the real system toward simulator-consistent dynamics during policy execution. This allows for the policy to operate in a state action regime closer to the Markov Decision Process distribution for which it was

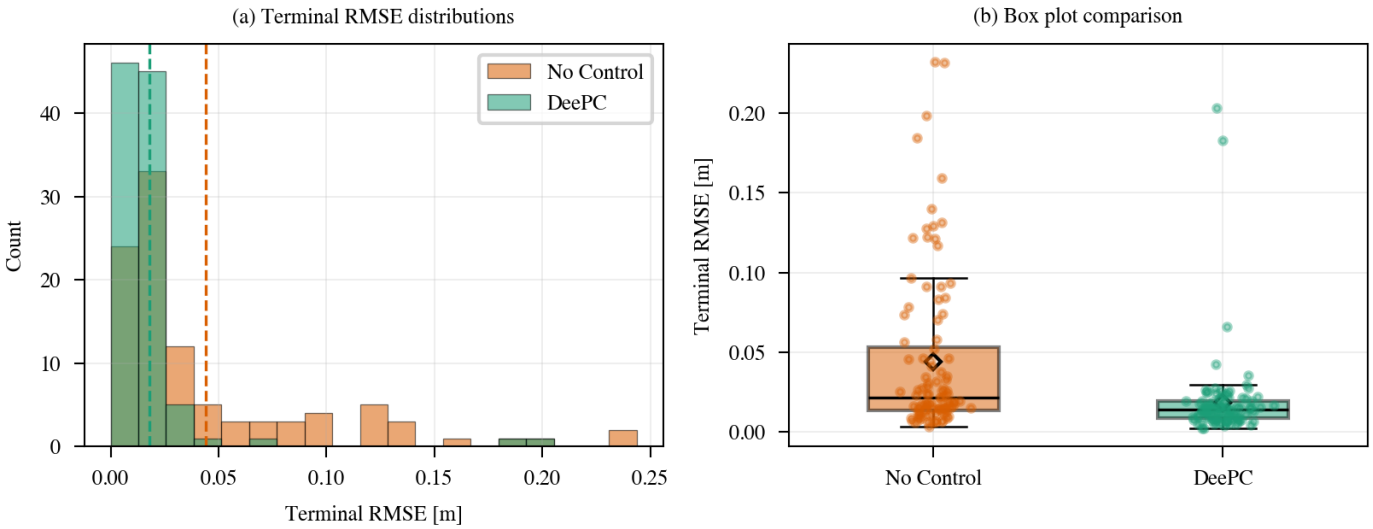


Fig. 4. Terminal RMSE distribution comparison

trained.

Experimental validation on a 7-DoF Franka Emika Panda robot demonstrated that this approach can significantly reduce the discrepancy between simulated and real trajectories. Across repeated reach tasks, the DeePC-assisted configuration achieved lower terminal pose errors and improved consistency compared to zero-shot policy transfer. While the average terminal error improved by approximately $2.6\,cm$, large deviations and policy misses could be mitigated with the proposed method. By operating closer to the state–action distribution for which the RL-policy was originally trained, accuracy, reliability, and transferability are massively improved.

While the reach task is fundamental for robotic manipulation, it does not capture additional effects that may contribute to the sim2real gap, such as synchronization discrepancies between simulated and physical enironment, object interaction dynamics or deformation effects. These factors can introduce more complex gap behavior that may deviate from the LTI assumption adopted in this work.

Several directions remain for future work. First, extending the approach to more complex manipulation tasks involving contacts, external disturbances, or object interaction would further evaluate its robustness in practical scenarios. In particular, tasks common in human–robot collaboration could benefit from the ability of DeePC to incorporate explicit safety constraints such as torque, velocity, or momentum limits.

Second, the current formulation relies on an LTI approximation of the sim2real gap. While effective in the presented experiments, nonlinear extensions or adaptive data updates could improve performance in more strongly nonlinear regimes or under time-varying dynamics.

Finally, improving computational efficiency and scaling the Hankel matrix formulation to higher-dimensional systems would enable higher control frequencies and broader applicability, such as high degree of freedom humanoids. Combining DeePC-based gap regulation with existing sim2real strategies—such as domain randomization or system identification—may also yield complementary benefits, potentially leading to more robust and scalable RL deployment pipelines for real-world robotics.

## APPENDIX

TABLE IV
DEEPC CONTROLLER PARAMETERS USED IN THE EXPERIMENTS.

| Parameter | Value |
|---|---|
| Prediction horizon $N$ | 15 |
| Past horizon $T_{ini}$ | 5 |
| Regularization $\lambda_g$ | ... |
| Regularization $\lambda_y$ | ... |
| Weight matrix $Q$ | ... |
| Weight matrix $R$ | ... |
| Solver | MOSEK |
| Update frequency | 10 Hz |

To compensate for the different magnitudes of the joint-gap signals, the cost matrix $Q$ incorporates inverse-variance scaling. Specifically,

$$Q = \mathrm{diag}\left(\frac{w_1}{\sigma_1^2}, \ldots, \frac{w_n}{\sigma_n^2}\right), \tag{10}$$

where $\sigma_i^2 = \mathrm{Var}(e_i)$ denotes the empirical variance of the $i$-th gap component and $w_i$ represents the corresponding weighting coefficient in the cost function.

## REFERENCES

[1] W. Zhao, J. P. Queralta, and T. Westerlund, "Sim-to-real transfer in deep reinforcement learning for robotics: a survey," in *2020 IEEE symposium series on computational intelligence (SSCI)*. IEEE, 2020, pp. 737–744.

[2] R. Tiwari, S. Khapre, and A. Singh, "Reinforcement learning in robotic systems: A review on sim-to-real transfer," *Robotics and Autonomous Systems*, p. 105327, 2026.

[3] S. Höfer, K. Bekris, A. Handa, J. C. Gamboa, M. Mozifian, F. Golemo, C. Atkeson, D. Fox, K. Goldberg, J. Leonard *et al.*, "Sim2real in robotics and automation: Applications and challenges," *IEEE transactions on automation science and engineering*, vol. 18, no. 2, pp. 398–400, 2021.

[4] O. M. Andrychowicz, B. Baker, M. Chociej, R. Jozefowicz, B. McGrew, J. Pachocki, A. Petron, M. Plappert, G. Powell, A. Ray *et al.*, "Learning dexterous in-hand manipulation," *The International Journal of Robotics Research*, vol. 39, no. 1, pp. 3–20, 2020.

[5] X. Chen, J. Hu, C. Jin, L. Li, and L. Wang, "Understanding domain randomization for sim-to-real transfer," *arXiv preprint arXiv:2110.03239*, 2021.

[6] J. Truong, S. Chernova, and D. Batra, "Bi-directional domain adaptation for sim2real transfer of embodied navigation agents," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 2634–2641, 2021.

[7] N. Sobanbabu, G. He, T. He, Y. Yang, and G. Shi, "Sampling-based system identification with active exploration for legged sim2real learning," in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, J. Lim, S. Song, and H.-W. Park, Eds., vol. 305. PMLR, 27–30 Sep 2025, pp. 578–598. [Online]. Available: https://proceedings.mlr.press/v305/sobanbabu25a.html

[8] X. Zhang, C. Wang, L. Sun, Z. Wu, X. Zhu, and M. Tomizuka, "Efficient sim-to-real transfer of contact-rich manipulation skills with online admittance residual learning," in *Conference on Robot Learning*. PMLR, 2023, pp. 1621–1639.

[9] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, "Residual reinforcement learning for robot control," *arXiv preprint arXiv:1812.03201*, 2018.

[10] G. Tiboni, P. Klink, J. Peters, T. Tommasi, C. D'Eramo, and G. Chalvatzaki, "Domain randomization via entropy maximization," *arXiv preprint arXiv:2311.01885*, 2023.

[11] J. Josifovski, M. Malmir, N. Klarmann, B. L. Žagar, N. Navarro-Guerrero, and A. Knoll, "Analysis of randomization effects on sim2real transfer in reinforcement learning for robotic manipulation tasks," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022, pp. 10 193–10 200.

[12] C. Tang, B. Abbatematteo, J. Hu, R. Chandra, R. Martín-Martín, and P. Stone, "Deep reinforcement learning for robotics: A survey of real-world successes," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 8, no. 1, pp. 153–188, 2025.

[13] S. James, P. Wohlhart, M. Kalakrishnan, D. Kalashnikov, A. Irpan, J. Ibarz, S. Levine, R. Hadsell, and K. Bousmalis, "Sim-to-real via sim-to-sim: Data-efficient robotic grasping via randomized-to-canonical adaptation networks," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 12 627–12 637.

[14] F. Muratore, F. Ramos, G. Turk, W. Yu, M. Gienger, and J. Peters, "Robot learning from randomized simulations: A review," *Frontiers in Robotics and AI*, vol. 9, p. 799893, 2022.

[15] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, May 2018, p. 3803–3810. [Online]. Available: http://dx.doi.org/10.1109/ICRA.2018.8460528

[16] L. Viano, Y.-T. Huang, P. Kamalaruban, A. Weller, and V. Cevher, "Robust inverse reinforcement learning under transition dynamics mismatch," 2021. [Online]. Available: https://arxiv.org/abs/2007.01174

[17] B. Tang, M. A. Lin, I. Akinola, A. Handa, G. S. Sukhatme, F. Ramos, D. Fox, and Y. Narang, "Industreal: Transferring contact-rich assembly tasks from simulation to reality," *arXiv preprint arXiv:2305.17110*, 2023.

[18] K. Wrede, S. Zarnack, R. Lange, O. Donath, T. Wohlfahrt, and U. Feldmann, "Curriculum design and sim2real transfer for reinforcement learning in robotic dual-arm assembly," *Machines*, vol. 12, no. 10, p. 682, 2024.

[19] A. A. Shahid, Y. S. Narang, V. Petrone, E. Ferrentino, A. Handa, D. Fox, M. Pavone, and L. Roveda, "Scaling population-based reinforcement learning with gpu accelerated simulation," *CoRR*, 2024.

[20] Y. Tian, J. Jacob, Y. Huang, J. Zhao, E. Gu, P. Ma, A. Zhang, F. Javid, B. Romero, S. Chitta *et al.*, "Fabrica: Dual-arm assembly of general multi-part objects via integrated planning and learning," *arXiv preprint arXiv:2506.05168*, 2025.

[21] B. Tang, I. Akinola, J. Xu, B. Wen, A. Handa, K. Van Wyk, D. Fox, G. S. Sukhatme, F. Ramos, and Y. Narang, "Automate: Specialist and generalist assembly policies over diverse geometries," *arXiv preprint arXiv:2407.08028*, vol. 1, no. 2, 2024.

[22] J. P. Allamaa, P. Patrinos, H. Van der Auweraer, and T. D. Son, "Sim2real for autonomous vehicle control using executable digital twin," *IFAC-PapersOnLine*, vol. 55, no. 24, pp. 385–391, 2022.

[23] J. Abou-Chakra, L. Sun, K. Rana, B. May, K. Schmeckpeper, M. Vittoria Minniti, and L. Herlant, "Real-is-sim: Bridging the sim-to-real gap with a dynamic digital twin for real-world robot policy evaluation," *arXiv e-prints*, pp. arXiv–2504, 2025.

[24] H. Li, X. He, Y. Wu, G. Liu, H. Wang, X. Wen, and L. Li, "Digital twin and ai-driven robotic embodied control system: a novel adaptive learning and decision optimization method," *Robotics and Computer-Integrated Manufacturing*, vol. 98, p. 103138, 2026.

[25] C. Samak, T. Samak, and V. Krovi, "A scalable and parallelizable digital twin framework for sustainable sim2real transition of multi-agent reinforcement learning systems. arxiv. abs/2403.1,(2024)."

[26] J. Coulson, J. Lygeros, and F. Dörfler, "Data-enabled predictive control: In the shallows of the deepc," in *2019 18th European control conference (ECC)*. IEEE, 2019, pp. 307–312.

[27] I. Iso, "Ts 15066: 2011: Robots and robotic devices collaborative robots," *Geneva, Switzerland: International Organization for Standardization, draft*, 2011.

[28] A. De Luca and F. Flacco, "Integrated control for phri: Collision avoidance, detection, reaction and collaboration," in *2012 4th IEEE RAS EMBS International Conference on Biomedical Robotics and Biomechatronics (BioRob)*, 2012, pp. 288–295.

[29] A. Carron, E. Arcari, M. Wermelinger, L. Hewing, M. Hutter, and M. N. Zeilinger, "Data-driven model predictive control for trajectory tracking with a robotic arm," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3758–3765, 2019.

[30] A. Waheed, M. Areti, L. Gallantree, and Z. Hasnain, "Quantifying the sim2real gap: Model-based verification and validation in autonomous ground systems," *IEEE Robotics and Automation Letters*, 2025.

[31] S. Haddadin, "The franka emika robot: A standard platform in robotics research," *IEEE Robotics Automation Magazine*, vol. PP, pp. 2–14, 12 2024.

[32] NVIDIA, :, M. Mittal, P. Roth, J. Tigue, A. Richard, O. Zhang, P. Du, A. Serrano-Muñoz, X. Yao, R. Zurbrügg, N. Rudin, L. Wawrzyniak, M. Rakhsha, A. Denzler, E. Heiden, A. Borovicka, O. Ahmed, I. Akinola, A. Anwar, M. T. Carlson, J. Y. Feng, A. Garg, R. Gasoto, L. Gulich, Y. Guo, M. Gussert, A. Hansen, M. Kulkarni, C. Li, W. Liu, V. Makoviychuk, G. Malczyk, H. Mazhar, M. Moghani, A. Murali, M. Noseworthy, A. Poddubny, N. Ratliff, W. Rehberg, C. Schwarke, R. Singh, J. L. Smith, B. Tang, R. Thaker, M. Trepte, K. V. Wyk, F. Yu, A. Millane, V. Ramasamy, R. Steiner, S. Subramanian, C. Volk, C. Chen, N. Jawale, A. V. Kuruttukulam, M. A. Lin, A. Mandlekar, K. Patzwaldt, J. Welsh, H. Zhao, F. Anes, J.-F. Lafleche, N. Moënne-Loccoz, S. Park, R. Stepinski, D. V. Gelder, C. Amevor, J. Carius, J. Chang, A. H. Chen, P. de Heras Ciechomski, G. Daviet, M. Mohajerani, J. von Muralt, V. Reutskyy, M. Sauter, S. Schirm, E. L. Shi, P. Terdiman, K. Vilella, T. Widmer, G. Yeoman, T. Chen, S. Grizan, C. Li, L. Li, C. Smith, R. Wiltz, K. Alexis, Y. Chang, D. Chu, L. J. Fan, F. Farshidian, A. Handa, S. Huang, M. Hutter, Y. Narang, S. Pouya, S. Sheng, Y. Zhu, M. Macklin, A. Moravanszky, P. Reist, Y. Guo, D. Hoeller, and G. State, "Isaac lab: A gpu-accelerated simulation framework for multi-modal robot learning," 2025. [Online]. Available: https://arxiv.org/abs/2511.04831

[33] S. Diamond and S. Boyd, "CVXPY: A Python-embedded modeling language for convex optimization," *Journal of Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.

[34] M. ApS, *The MOSEK Python Fusion API manual. Version 11.1.8*, 2025. [Online]. Available: https://docs.mosek.com/latest/pythonfusion/index.html